In the preceding chapters, the agent interacts with its environment while learning: it tries a price, observes a purchase, and updates its belief. Online learning is natural in digital markets where experimentation is cheap and feedback is instant. In many economically important settings, however, experimentation is impossible or prohibitively costly. A hospital cannot randomly assign treatments to learn optimal dosing. A central bank cannot experiment with interest rate schedules to discover optimal monetary policy. A firm inheriting a decade of transaction logs wants to improve its pricing rule without conducting new experiments during the transition. In each case, the agent has access to a fixed dataset of past decisions and outcomes, collected under some historical policy, and must learn the best possible new policy from this data alone.

This is the problem of *offline reinforcement learning*, also called *batch reinforcement learning*.[1] The agent never queries the environment. All learning happens from a fixed dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^n$ collected by some behavioral policy π_b. The goal is to find a policy $\hat{\pi}$ whose value $V^{\hat{\pi}}$ is as close to the optimal V^* as possible.

Online RL algorithms (Q-learning, SARSA, policy gradient methods from Section **??**) can in principle be applied to offline data by treating the dataset as a replay buffer. In practice, this fails catastrophically. The reason is *distributional shift*: the learned policy $\hat{\pi}$ inevitably queries state-action pairs that the behavioral policy π_b never visited, and the Q-function at these unseen pairs is pure extrapolation. Because the Bellman backup propagates these extrapolation errors through the max operator, errors compound geometrically across the planning horizon, producing arbitrarily poor policies.[2]

0.1 The Pessimism Principle

The overestimation failure has a clean theoretical characterization. Consider tabular Q-learning with a fixed dataset. At any state-action pair (s, a) not in the dataset, the empirical Bellman backup is undefined, but the max in $\max_{a'} \hat{Q}(s', a')$ may still select it if the randomly-initialized Q-value happens to be high. With function approximation, the problem is subtler but identical in spirit: the function approximator can assign high

[1] The term "batch RL" was standard in the earlier literature (Ernst et al., 2005; Lange et al., 2012). "Offline RL" became dominant after Levine et al. (2020), who distinguished it from off-policy RL (which still collects new data, just under a different policy than the target). I use "offline RL" throughout.

[2] This failure mode was first demonstrated empirically by Fujimoto et al. (2019), who showed that standard off-policy algorithms (DDPG, SAC) trained purely from a static dataset performed worse than the behavioral policy itself, even when that behavioral policy was a partially-trained, mediocre agent. The gap widened with dataset size, the opposite of what one expects from more data.

values to out-of-distribution inputs without any corrective signal.

The solution, established simultaneously by several groups, is the *pessimism principle*: construct a lower confidence bound on the Q-function and optimize against it. Formally, given a dataset $\mathcal{D}$ and a confidence parameter δ, construct a penalty function $\Gamma(s, a)$ that is large where data coverage is poor, and define the pessimistic Q-function

$$\tilde{Q}(s, a) = \hat{Q}(s, a) - \Gamma(s, a) \tag{1}$$

where $\hat{Q}$ is the standard empirical Bellman solution. The policy $\hat{\pi}(s) = \arg\max_a \tilde{Q}(s, a)$ selects actions that are both high-value *and* well-supported by data.

Definition D1 (Pessimistic Value Iteration, PEVI (Jin et al., 2021))**.** *Given dataset $\mathcal{D}$, penalty function $\Gamma_h(s, a)$ for each stage h, and horizon H, PEVI computes*

$$\hat{Q}_h(s, a) = r_h(s, a) + \hat{P}_h \hat{V}_{h+1}(s, a) - \Gamma_h(s, a) \tag{2}$$

$$\hat{V}_h(s) = \max\{\max_a \hat{Q}_h(s, a),\, 0\} \tag{3}$$

$$\hat{\pi}_h(s) = \arg\max_a \hat{Q}_h(s, a) \tag{4}$$

where $\hat{P}_h$ is the empirical transition operator estimated from $\mathcal{D}$.

Jin et al. (2021) show that with $\Gamma_h(s, a) = c \cdot \sqrt{H^3/N_h(s, a)}$, where $N_h(s, a)$ counts visits to (s, a) at stage h and c is an absolute constant, PEVI achieves

$$V^* - V^{\hat{\pi}} \leq \tilde{O}\left(\sum_{h=1}^{H} \mathbb{E}_{\pi^*}\left[\sqrt{\frac{1}{N_h(s_h, a_h)}}\right]\right) \tag{5}$$

with high probability. The bound depends on the data coverage at states and actions visited by the *optimal* policy π^*, not the full state-action space. This is the key advantage of pessimism over uniform coverage requirements.

0.1.1 Concentrability and Coverage

The bound in (5) is instance-dependent, scaling with how well π_b covers π^*. The classical way to formalize this is through *concentrability coefficients* (Munos and Szepesvári, 2008).

Definition D2 (Single-policy concentrability)**.** *The single-policy concentrability coeffi-*

cient of π^* with respect to π_b is

$$C^* = \max_{h \in [H]} \left\| \frac{d_h^{\pi^*}}{d_h^{\pi_b}} \right\|_\infty \tag{6}$$

where $d_h^\pi(s, a)$ is the state-action occupancy measure of π at stage h.

When C^* is finite, the optimal policy visits only states and actions that π_b also visits with non-negligible probability, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (5) remain controlled. Rashidinejad et al. (2021) prove that single-policy concentrability is both necessary and sufficient for offline learning: if $C^* < \infty$, then $O(|\mathcal{S}|H^2C^*/\epsilon^2)$ samples suffice for an ϵ-optimal policy; if $C^* = \infty$, no algorithm can guarantee suboptimality better than the behavioral policy.

0.1.2 Impossibility Results

Zanette (2021) establish fundamental limits on offline RL by showing that it can be exponentially harder than online RL. Specifically, there exist MDPs with S states and horizon H where online RL finds an ϵ-optimal policy in $\text{poly}(S, H, 1/\epsilon)$ episodes, but any offline algorithm requires $\Omega(2^H)$ samples unless the dataset covers all reachable states. The construction uses a binary tree MDP where the optimal path visits a unique leaf, and any dataset that misses this leaf provides no information about the optimal action at the root.

The practical implication is that offline RL is not a universal replacement for online experimentation. When the behavioral policy is far from optimal, especially in long-horizon problems, offline methods provably cannot recover the optimal policy without exponentially large datasets. Pessimistic algorithms are the best one can do, but they are still fundamentally constrained by what the data contains.

0.2 Algorithms

I present four practical algorithms that instantiate different approaches to the distributional shift problem. All four can be understood as modifications of standard Q-learning (Section **??**) that prevent the agent from overvaluing actions outside the data support.

0.2.1 Fitted Q-Iteration

Fitted Q-Iteration (FQI, Ernst et al., 2005) is the simplest offline RL algorithm, predating the modern pessimism framework. FQI applies the standard Bellman backup iteratively using supervised regression on the fixed dataset, as described in Section **??**. It does not include any explicit pessimism mechanism. When state-action coverage is poor, the max in the Bellman backup selects the highest Q-value among all actions at s', including actions never observed in the data. If the function approximator generalizes poorly at these unseen actions, targets become noisy and biased upward, causing the overestimation cascade. FQI works well when coverage is good but degrades as coverage gaps grow.[3]

0.2.2 Conservative Q-Learning

Conservative Q-Learning (CQL, Kumar et al., 2020) adds an explicit penalty that pushes down Q-values at actions not well-represented in the data. The key idea is to add a regularizer to the Bellman error objective that minimizes Q-values under a broad distribution over actions while maximizing Q-values at the actions actually taken in the dataset.

Definition D3 (Conservative Q-Learning)**.** *CQL modifies the standard Bellman error objective by adding a conservative regularizer. At each iteration, CQL solves*

$$\hat{Q}_{k+1} = \arg\min_{Q} \ \alpha \left(\mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_{a} \exp Q(s,a) \right] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)] \right) + \frac{1}{2}\mathbb{E}_{\mathcal{D}} \left[(Q(s,a) - \hat{\mathcal{B}}^{\pi_k}\hat{Q}_k(s,a))^2 \right] \tag{7}$$

where $\alpha > 0$ is a hyperparameter controlling the degree of conservatism, and $\hat{\mathcal{B}}^{\pi_k}$ is the empirical Bellman operator.

The first term in the regularizer, $\log \sum_a \exp Q(s,a)$, is a soft maximum over all actions, pushing down Q-values uniformly. The second term, $\mathbb{E}_{\mathcal{D}}[Q(s,a)]$, pulls Q-values back up at the data actions. The net effect is a penalty on Q-values for actions that appear infrequently relative to their softmax contribution. Kumar et al. (2020) prove that the resulting Q-function is a pointwise lower bound on the true Q-function (Theorem 3.2), making it a concrete instantiation of the pessimism principle from (1) with an implicit, data-adaptive penalty Γ.

[3] Munos and Szepesvári (2008) prove finite-time error bounds for FQI under approximate Bellman completeness and all-policy concentrability. Both assumptions are strong, and violation of either leads to divergence in practice.

0.2.3 Implicit Q-Learning

Implicit Q-Learning (IQL, Kostrikov et al., 2022) avoids querying Q-values at unseen actions entirely. Instead of computing $\max_{a'} Q(s', a')$ in the Bellman backup (which requires evaluating Q at potentially out-of-distribution actions), IQL learns a separate value function $V(s)$ that approximates the in-sample maximum through *expectile regression*.

Definition D4 (Implicit Q-Learning)**.** *IQL maintains three functions: $Q_\theta(s, a)$, $V_\psi(s)$, and a policy $\pi_\phi(a|s)$. The value function is trained via expectile regression*

$$L_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[L_2^\tau(Q_{\bar{\theta}}(s, a) - V_\psi(s)) \right] \tag{8}$$

where $L_2^\tau(u) = |\tau - \mathbb{1}\{u < 0\}| \cdot u^2$ is the asymmetric squared loss with expectile parameter $\tau \in (0.5, 1)$, and $Q_{\bar{\theta}}$ uses a target network. The Q-function is trained with V_ψ as the continuation value

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(r + \gamma V_\psi(s') - Q_\theta(s, a))^2 \right] \tag{9}$$

The expectile parameter τ controls the degree of optimism within the data support. As $\tau \to 1$, the expectile converges to the in-sample maximum; as $\tau \to 0.5$, it converges to the in-sample mean. Setting $\tau = 0.7$ balances exploiting the best observed actions against the noise in finite samples. The critical property is that the Q-function is never evaluated at actions outside the dataset: the max operation is implicit in the expectile regression of V.

0.2.4 Batch-Constrained Q-Learning

Batch-Constrained Q-Learning (BCQ, Fujimoto et al., 2019) takes a different approach: rather than modifying the Q-function objective, it restricts the policy to actions similar to those in the dataset. BCQ first learns a generative model $G_\omega(s)$ of the behavioral policy, then constrains the policy to only select actions with high likelihood under G_ω.

Definition D5 (Batch-Constrained Q-Learning)**.** *BCQ learns a behavioral model $G_\omega(a|s)$ from the dataset via maximum likelihood, and constrains action selection*

$$\hat{\pi}(s) = \arg \max_{a: G_\omega(a|s) \geq \tau \cdot \max_{a'} G_\omega(a'|s)} Q_\theta(s, a) \tag{10}$$

where $\tau \in (0, 1]$ is a threshold parameter. The Q-function is trained with standard Bellman backups, but the max in the target computation is also constrained to the feasible action set.

BCQ's constraint is hard rather than soft: actions with behavioral probability below τ times the most likely action are simply excluded from consideration. This prevents the Q-function from ever being queried at truly out-of-distribution actions, addressing the distributional shift problem at the policy level rather than the value function level. The tradeoff is that BCQ's performance is bounded by the quality of actions in the dataset. If the behavioral policy never takes the optimal action at some state, BCQ cannot discover it regardless of sample size.

0.3 Simulation: Offline RL for Dynamic Pricing

The simulation study evaluates the four offline RL algorithms on a perishable inventory pricing problem with demand regime switching. A retailer with $I_{\max} = 30$ units of perishable inventory must set prices over $H = 20$ periods. The state (i, d, t) consists of current inventory $i \in \{0, \ldots, 30\}$, demand regime $d \in \{1, 2, 3, 4\}$, and time remaining $t \in \{1, \ldots, 20\}$. The action is a price $p \in \{1, \ldots, 10\}$. Demand follows $Q \sim \text{Poisson}(\lambda_0[d] \cdot e^{-0.15p})$ with base rates $\lambda_0 = (1.5, 3.0, 5.0, 8.0)$, and the reward is $r = p \cdot \min(Q, i)$. Demand regimes follow a 4-state Markov chain with diagonal persistence 0.6. Unsold inventory at the terminal period incurs a spoilage cost of \$2.00 per unit, making clearance pricing valuable near the deadline.[4] The behavioral policy represents a conservative pricing team that always sets the maximum price ($p = 10$) regardless of demand regime, inventory, or time remaining, with probability 0.85 and randomizes uniformly over all prices with probability 0.15.[5] All episodes start at full inventory ($i = 30$). All offline methods train on 500 episodes and are evaluated over 1,000 episodes against

[4]The spoilage penalty creates distributional shift. The optimal policy adapts prices to inventory level and time remaining, using lower prices near the deadline when inventory is high. The behavioral policy ignores these state variables and prices at the maximum, so the Q-function at state-adapted pricing actions is extrapolation from sparse data. The \$2.00 penalty is a deliberate design choice: under harsher penalties (e.g., \$10 per unit), all methods collapse to 48–53% of optimal regardless of algorithmic sophistication, confirming that no offline correction can overcome severe distributional shift when the penalty regime amplifies consequences of the behavioral policy's suboptimality.

[5]With 500 episodes (10,000 transitions) on a state-action space of 24,800 pairs, the 85% concentration at price 10 ensures that the behavioral state-action occupancy diverges significantly from the optimal policy's occupancy, while the 15% uniform component provides sparse off-policy coverage.

Table 1: Policy value for each offline RL method, expressed as mean return and percentage of the DP optimal. Standard errors computed over 20 seeds.

Method	Mean Return	% of Optimal
DP Oracle	192.41 ± 0.33	100.0%
BC	169.27 ± 0.60	88.0%
FQI	156.18 ± 1.68	81.2%
CQL	176.73 ± 1.13	91.9%
IQL	176.98 ± 0.56	92.0%
BCQ	169.27 ± 0.60	88.0%

the DP optimal policy computed by backward induction.[6] Results are averaged over 20 independent seeds.

FQI achieves 81.2% of the DP optimal, substantially below the behavioral cloning baseline of 88.0% (Table 1). Without any mechanism to control extrapolation error, the $\max_{a'} Q(s', a')$ operator in FQI's Bellman backup selects overestimated Q-values at out-of-distribution actions, and these overestimates compound across 200 iterations of fitted Q-iteration. CQL and IQL both exceed the behavioral baseline, achieving 91.9% and 92.0% respectively.[7] CQL's conservative penalty suppresses Q-values at actions not well-represented in the data while preserving relative ordering among data-supported actions, allowing the policy to discover lower prices that clear inventory near the deadline. IQL achieves the same improvement through a different mechanism: by replacing $\max_{a'} Q(s', a')$ with the expectile-regressed value function $V(s')$ as the Bellman continuation, IQL avoids querying Q at unseen actions during training while still extracting a policy that improves on the behavioral at states where the 15% noise component revealed better pricing actions. BCQ matches the behavioral at 88.0%; its action constraint restricts the policy to prices near 10, preventing both overestimation and improvement.[8]

These results validate several theoretical predictions from the preceding sections. FQI's degradation below the behavioral baseline (81.2% versus 88.0%) demonstrates the

[6] FQI uses the standard Bellman backup with $\max_{a'} Q(s', a')$, deliberately without a target network, to isolate the overestimation cascade as a pedagogical baseline. Adding target networks to FQI mitigates but does not eliminate extrapolation error. CQL and IQL include target networks following their original implementations (Kumar et al., 2020; Kostrikov et al., 2022); for CQL, target networks proved essential, as the conservative penalty amplifies bootstrap instability without them.

[7] CQL uses $\alpha = 0.1$, the result of a search over $\alpha \in \{5.0, 2.0, 0.5, 0.1\}$. Larger values push Q-values down too aggressively, collapsing the learned policy to the behavioral action at most states; the right α is problem-specific and can vary by orders of magnitude.

[8] When the behavioral policy concentrates 85% probability at a single action, BCQ's threshold constraint permits only that action at most states, effectively reducing BCQ to behavioral cloning regardless of the learned Q-values.

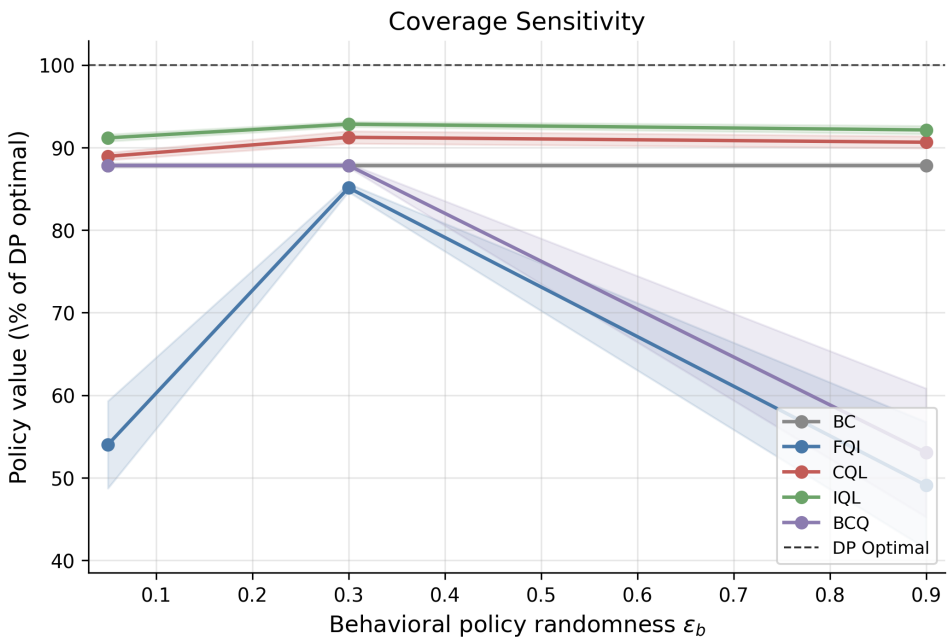

Figure 1: Policy value (as % of DP optimal) versus behavioral policy randomness ϵ_b for four offline RL methods and the behavioral cloning baseline (BC). Higher ϵ_b increases data coverage. FQI peaks at moderate coverage ($\epsilon_b = 0.3$) and collapses at both extremes. BCQ degrades at high ϵ_b as its behavioral constraint becomes vacuous.

overestimation cascade described in Section 0.2: without pessimism, the $\max_{a'}$ operator selects overestimated Q-values at out-of-distribution actions, and 200 iterations of bootstrapping compound these errors geometrically (Fujimoto et al., 2019). CQL and IQL both exceeding BC confirms the pessimism principle (Section 0.1): CQL's conservative penalty produces a pointwise lower bound on the true Q-function (Definition D3, Theorem 3.2 of Kumar et al. 2020), while IQL's expectile regression (Definition D4) avoids querying Q at unseen actions entirely (Kostrikov et al., 2022). BCQ matching BC exactly illustrates the action-constraint tradeoff formalized in Definition D5: performance is bounded by the quality of actions in the data, and when the behavioral policy concentrates on a single action, no amount of Q-learning can overcome the constraint.

Figure 1 varies the behavioral noise parameter $\epsilon_b \in \{0.05, 0.3, 0.9\}$. CQL and IQL remain above the BC baseline across all coverage levels, confirming that both pessimism mechanisms provide robustness to the data distribution. FQI exhibits non-monotone behavior: it peaks at moderate coverage ($\epsilon_b = 0.3$) where partial exploration controls the overestimation cascade, but collapses at both extremes.[9] BCQ collapses at $\epsilon_b = 0.9$ be-

[9] At high ϵ_b, the near-uniform behavioral policy provides Q-function targets across all actions, giving the unconstrained $\max_{a'}$ operator more opportunities to select overestimated values rather than fewer.

cause a nearly uniform behavioral policy renders the action constraint vacuous, reducing BCQ to unconstrained FQI. These coverage patterns connect directly to the concentrability framework (Definition D2): as ϵ_b decreases, the concentrability coefficient C^* grows because the optimal policy's state-action occupancy diverges from the behavioral policy's, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (5) become large at states the optimal policy visits. CQL and IQL remain robust because their conservative adjustments scale with data sparsity, while FQI lacks this adaptive correction.

0.4 From Offline RL to Human Feedback

The offline RL algorithms above assume access to a scalar reward signal in the dataset. When rewards are observed, the problem reduces to learning a good policy from fixed data. In many domains, however, the reward itself is unknown and must be learned from human judgments. Reinforcement Learning from Human Feedback (RLHF) combines offline preference data with the policy optimization tools of offline RL: the agent never interacts with the environment during training, and all learning proceeds from a static dataset of human comparisons. The methods below extend the offline RL framework from learning policies given rewards to learning rewards given preferences, and then optimizing policies against those learned rewards.

Every chapter so far assumes access to a scalar reward signal: dynamic programming requires $r(s, a)$, model-free RL observes r_t after each transition, and the Bellman optimality equation presupposes that rewards are known or observable. When they are neither, the DP/RL machinery cannot be applied directly.

In many domains, scalar rewards are unavailable but ordinal preferences over trajectories are easy to elicit. A human evaluator cannot assign a meaningful numerical score to a paragraph of text, but can reliably say "response A is better than response B." The raw data is a set of trajectory pairs with binary preference labels. RLHF uses these ordinal comparisons to learn a proxy reward function r_θ, which then serves as the scalar signal for standard RL optimization. This is not an inverse problem in the IRL sense; the goal is not to rationalize observed behavior, but rather a two-stage forward problem in which the analyst first learns a reward from human judgments and then solves the resulting MDP. Christiano et al. (2017) demonstrated that this approach could train agents without an explicit reward function. RLHF has since become the predominant method for aligning

large language models.

0.5 Learning Rewards from Preferences

The canonical RLHF framework is built on a formal model of human preference. The observed data consists of tuples (s, y_w, y_l), where s is a context, and y_w and y_l are two outputs, with y_w being the "winner" preferred by a human. Assuming preferences follow a latent utility model, the Bradley-Terry model (Bradley and Terry, 1952) gives the probability that y_w is preferred: $P(y_w \succ y_l|s) = \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))$, where $r_\theta : \mathcal{S} \times \mathcal{Y} \to \mathbb{R}$ is a learned *reward model* parameterized by θ, trained to approximate human preferences (not the ground-truth reward, which is unobserved), and $\sigma(\cdot)$ is the logistic function. This formulation is a binary logit model (Section **??**, Equation **??**).[10] The reward model parameters θ are estimated by minimizing the negative log-likelihood of the observed human choices:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(r_\theta(s, y_w) - r_\theta(s, y_l)\right)\right], \tag{11}$$

In the LLM setting, the "outputs" y_w and y_l are token sequences, i.e., trajectories of the autoregressive policy[11], so preferences are over trajectories rather than single actions. The preference loss in Equation (11) is MLE of a choice model where the "alternatives" are trajectory segments and the "choice" is the human-preferred one.

0.6 The RLHF Pipeline and Direct Optimization

The learned r_θ then serves as a proxy objective for policy optimization. Building on the reward-learning and RL fine-tuning framework of Ziegler et al. (2019) and Stiennon et al. (2020), the canonical three-stage pipeline was formalized by Ouyang et al. (2022). First, a base pretrained model is initialized via supervised fine-tuning (SFT)[12] on a small dataset of high-quality demonstrations, yielding an initial policy π^{SFT}. This grounds the

[10] Iskhakov et al. (2020) discuss the contrasts and synergies between machine learning and structural econometrics, including the shared reliance on logit-based choice models that underlies both RLHF and dynamic discrete choice estimation.

[11] An *autoregressive* language model generates text token by token: at each step it outputs a distribution over the vocabulary conditioned on all preceding tokens, then samples the next token; the full response is therefore a trajectory in token space and the model acts as a sequential policy over a vocabulary-sized action set.

[12] *Pretraining* optimizes a language model to predict the next token across a massive text corpus, producing broad linguistic knowledge with no behavioral objective. *Supervised fine-tuning* (SFT) continues training on a small curated dataset of (prompt, ideal-response) pairs to specialize the model toward the desired task and establish the reference policy π^{SFT} from which the KL penalty is measured.

model in the desired style and format. The second step is the reward model training as described, using preference data generated from this π^{SFT}.

In the third step, the SFT policy π_ϕ is fine-tuned via PPO (Section **??**) to maximize the frozen reward model r_θ, with a KL-divergence penalty preventing the policy from drifting into regions where r_θ is unreliable. The objective is

$$J(\phi) = \mathbb{E}_{s\sim\mathcal{D}, y\sim\pi_\phi(\cdot|s)}[r_\theta(s,y)] - \lambda_{KL}\mathbb{E}_{s\sim\mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s)\,||\,\pi^{SFT}(\cdot|s))], \tag{12}$$

where D_{KL} is the Kullback-Leibler divergence and λ_{KL} controls the penalty strength.[13] Without this constraint, the policy exploits inaccuracies in r_θ to achieve high proxy scores with degenerate behavior ("reward hacking"), the RLHF analogue of divergence under function approximation (Section **??**).

The KL-regularized objective in Equation (12) admits a Bayesian interpretation (Korbak et al., 2022). In this view, the reference policy π^{SFT} acts as a prior distribution over plausible responses. The reward model r_θ provides evidence, specifying which responses are more desirable. The goal of alignment is to find the posterior distribution π^* that optimally combines the prior with this evidence. This ideal posterior policy takes the form of Equation (13):

$$\pi^*(y|s) \propto \pi^{SFT}(y|s)\exp\left(\frac{r_\theta(s,y)}{\lambda_{KL}}\right). \tag{13}$$

The reward function scaled by λ_{KL} defines the log-likelihood, so the KL-regularized objective $J(\phi)$ is equivalent (up to an additive constant) to the Evidence Lower Bound (ELBO) for this Bayesian inference problem. Maximizing the RLHF objective via PPO is therefore variational inference: finding the policy π_ϕ that minimizes KL divergence to π^*. This reframes the KL penalty as a structural component of the inference problem rather than an ad-hoc regularizer.

Despite this closed-form characterization, the three-stage RLHF pipeline is complex to implement, requiring training multiple large models and a computationally expensive RL loop. Direct Preference Optimization (DPO), introduced by Rafailov et al. (2023), collapses the pipeline into a single supervised learning objective by reparameterizing the

[13] λ_{KL} denotes the KL penalty weight, reserving β for model parameters and γ for discount factors. The standard RLHF literature, including Rafailov et al. (2023), uses β for this parameter.

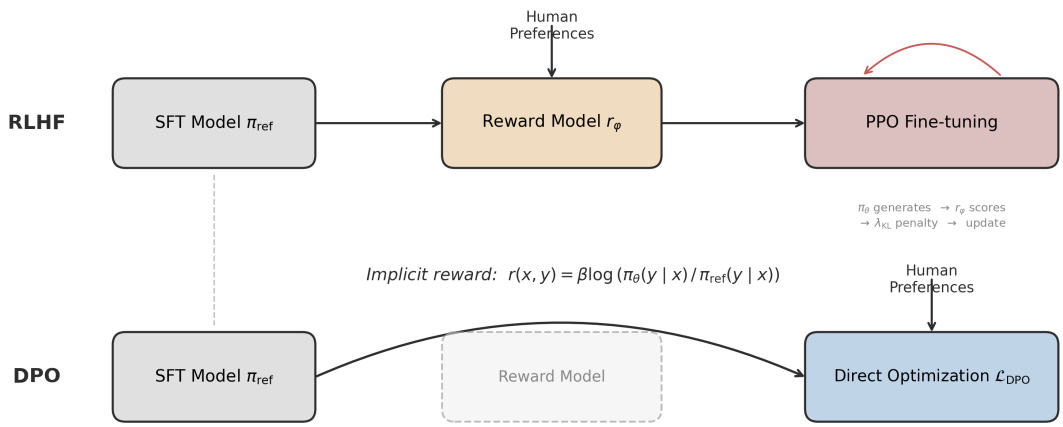

Figure 2: RLHF versus DPO pipelines. Top row: the three-stage RLHF pipeline trains a reward model from human preferences, then uses PPO to fine-tune the policy with a KL penalty. Bottom row: DPO collapses the pipeline into a single supervised learning objective over preference pairs, eliminating the explicit reward model (ghosted box).

reward function in terms of π^* and π^{SFT}, as in Equation (14):

$$r(s, y) = \lambda_{KL} \log \left(\frac{\pi^*(y|s)}{\pi^{SFT}(y|s)} \right) + \lambda_{KL} \log Z(s). \tag{14}$$

When this analytical expression for the reward is substituted into the Bradley-Terry preference loss from Equation (11), the unknown partition function $Z(s)$ cancels out. This yields a loss function that depends only on the policy π_ϕ being optimized and the fixed reference policy π^{SFT}. The DPO objective, given in Equation (15), is thus a simple binary cross-entropy loss over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi; \pi^{SFT}) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\lambda_{KL} \log \frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \lambda_{KL} \log \frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)} \right) \right]. \tag{15}$$

This objective is optimized on ϕ using standard supervised learning with a static preference dataset. The gradient increases the likelihood of preferred responses y_w and decreases the likelihood of dispreferred responses y_l, relative to the reference policy.

0.7 Recent Developments

A subsequent innovation leverages DPO to create an iterative self-improvement loop, reducing reliance on static, human-annotated data (Yuan et al., 2024). In this paradigm, dubbed Self-Rewarding Language Models, a single language model acts as both the instruction-following agent and its own reward model. The process begins with a base model fine-tuned to have both instruction-following and evaluation capabilities ("LLM-as-a-Judge"). In each subsequent iteration, the current model generates a new preference dataset for itself by producing multiple responses to a set of prompts and then evaluating its own outputs to assign scores. These scores are used to construct preference pairs (y_w, y_l), which form an AI-generated feedback dataset. The model is then fine-tuned on this new data using the DPO loss. This iterative process creates a feedback loop where enhancements in instruction-following ability lead to better reward modeling, which in turn fuels the next round of policy optimization.

While RLHF and its successors have proven effective at aligning model behavior with human preferences, several open issues remain. The framework does not aim to solve the identification problem in the strict econometric sense; the learned reward model r_θ (whether explicit or implicit in DPO) is a proxy for preference, not necessarily the uniquely identified "true" utility function required for welfare analysis.[14] Furthermore, the notion of "human feedback" is a significant simplification, as the preferences being optimized are those of a small, non-representative group of paid labelers, raising important questions about whose values are being embedded in these systems. Finally, the fine-tuning process can lead to degraded performance on standard academic benchmarks, a phenomenon called the *alignment tax* (Ouyang et al., 2022).[15] Mitigating these challenges while scaling alignment beyond the limits of human data collection remains a key frontier for the field.

0.8 Simulation Study: Preference Learning in Job Search

A worker searches for jobs in a labor market with compensating differentials, following a McCall (1970)-style search model. Each job is characterized by a wage $w \in$

[14] The identification problem in preference learning mirrors that in discrete choice. The reward function is identified only up to an additive constant and requires a location normalization.

[15] The alignment tax, as described by Ouyang et al. (2022), refers to performance regressions on public NLP benchmarks (SQuAD, DROP, HellaSwag) that result from RLHF fine-tuning.

$\{20, 28, 38, 50, 65, 82, 100, 125\}$ (thousands) and an amenity level $z \in \{0, 1, \ldots, 6\}$ capturing commute quality, flexibility, and job security. The state space has 112 states: 56 searching states in which the worker observes a pending offer (w_i, z_j) and decides to accept or reject, plus 56 employed states (w_i, z_j) in which the worker decides to stay or quit. The offer distribution exhibits compensating differentials, with wage rank and amenity rank negatively correlated ($\rho = -0.74$): high-wage offers cluster with low amenities and vice versa. The worker's true per-period utility is $u(w, z) = \alpha \log(w) + (1 - \alpha)z$ with $\alpha = 0.6$, but this function is unobserved; the worker can only compare career trajectories ("I prefer path A to path B"), exactly as in stated-preference surveys in labor economics. While searching, the worker receives the unemployment benefit $u_b = \alpha \log(b)$ where $b = 28$. Layoffs occur with probability $p = 0.05$ per period, and the discount factor is $\gamma = 0.95$. Dynamic programming gives $V^*(s_0) = 74.13$; the optimal policy accepts 25 of 56 offer types and stays employed at 25 of 56 job types. Preference data is generated by rolling out a uniform random policy from a random searching state. Each rollout produces a career segment of $L = 15$ periods recording states and actions. For each of K comparisons, two independent career segments are generated; the segment with higher cumulative discounted utility under the true (unobserved) utility function is labeled as preferred via the Bradley-Terry model. Figure 3 shows the optimal accept/reject and stay/quit boundaries.

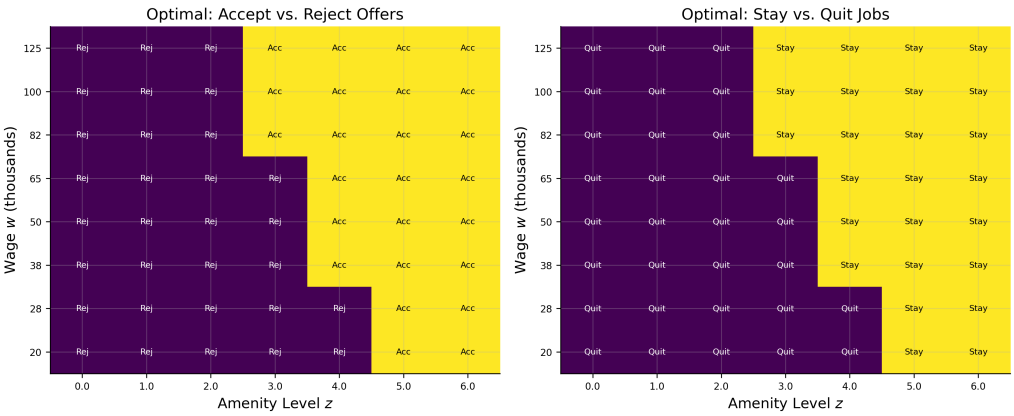

Figure 3: Left: the optimal accept/reject boundary for searching states in wage-amenity space. Right: the optimal stay/quit boundary for employed states. Both boundaries cut diagonally, reflecting the tradeoff between wage and amenity quality under compensating differentials.

Six methods are compared across $K \in \{25, 50, 100, 200, 500, 1{,}000, 2{,}000, 5{,}000\}$, averaged over 30 seeds. The neural network RLHF reward model is a two-layer MLP trained

by Bradley-Terry MLE on the K segment pairs; per-transition rewards are discount-weighted and summed to obtain a segment score, and the resulting 112-state reward table is solved by value iteration.[16] The correctly specified structural model parameterizes utility as $\hat{u} = \hat{\alpha}\log(w) + (1-\hat{\alpha})z$, estimating the single parameter $\hat{\alpha}$ via Bradley-Terry MLE, then solves the MDP. The misspecified model uses $\hat{u} = \hat{\alpha}\log(w) + (1-\hat{\alpha})\bar{z}$, where $\bar{z} = 3$ is the mean amenity level; this model ignores amenity variation across jobs, treating all amenities as identical. DPO trains a tabular softmax policy directly from trajectory comparisons, bypassing reward modeling entirely.[17] Tabular Q-learning (10,000 episodes, $\varepsilon = 0.15$, learning rate 0.1) and exact DP provide scalar-reward baselines. All four preference methods receive identical comparison data per seed; DPO receives a same-state variant generated from the same seed.

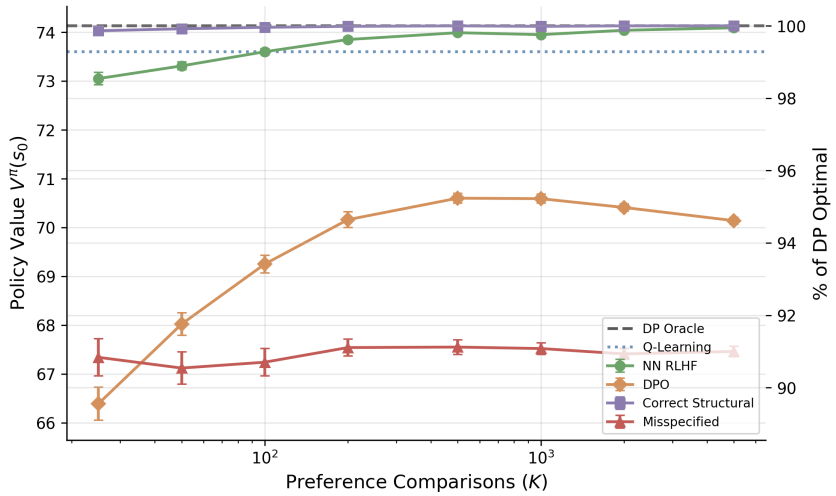

Figure 4: Policy value $V^{\pi}(s_0)$ versus number of preference comparisons K for all six methods (30 seeds, $L = 15$). The right axis shows the percentage of DP-optimal value.

Figure 4 reports the main results. Three findings stand out. First, the correctly specified structural model dominates: even at $K = 25$ it achieves 99.9% of DP-optimal, illustrating the power of correct specification when the model has a single free parameter. Second, the neural network converges more slowly but reaches 99.9% by $K = 5{,}000$, reflecting the higher sample complexity of a flexible model relative to a one-parameter

[16] The neural network has 4 inputs (normalized log-wage, normalized amenity, employment indicator, action), 32 hidden units per layer, and ∼1,200 parameters. The logistic loss from Equation (11) is applied to discount-weighted segment reward sums.

[17] DPO uses 112 logit parameters ϕ_s, one per state, trained via the DPO loss (Equation 15) with Adam optimization over a sweep of $\lambda_{KL} \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0\}$, selecting the λ_{KL} that minimizes training loss. The reference policy is uniform: $\pi^{SFT}(a|s) = 0.5$. To match the LLM setup where both completions condition on the same prompt, DPO comparison pairs start from the same initial state.

structural specification. Third, DPO plateaus at approximately 95% by $K = 500$ and does not improve further.[18] This plateau is qualitatively different from the gridworld failure (−118%): DPO recovers a reasonable policy, but the gap does not close with additional data.[19] The misspecified constant-amenity model plateaus at 91% regardless of K, because additional preference data cannot correct the omitted variable.

Table 2: Diagnostics at $K = 5{,}000$ (single seed): policy agreement with π^*, value-function correlation, and mean accepted wage and amenity for each method.

Method	Policy agree. (%)	V^π corr.	Mean amenity	Mean wage	$\hat{\alpha}$
NN RLHF	96.4	1.000	4.67	73	—
DPO	57.1	0.777	3.38	63	—
Correct	100.0	1.000	4.84	71	0.597
Misspecified	50.0	0.889	3.00	70	—
Optimal	100.0	1.000	4.84	71	—

Table 2 provides state-level diagnostics at $K = 5{,}000$. The structural model and neural network achieve near-perfect policy agreement with π^* (100% and 96% respectively). DPO agrees on only 57% of states with value-function correlation 0.78, systematically underselecting on both wage and amenity dimensions.[20] The misspecified model agrees on 50%, with disagreements concentrated where amenity variation matters.[21]

Figure 5 reports the segment length ablation at $K = 2{,}000$. At $L = 1$, both methods perform poorly because single-step comparisons carry minimal information about long-run value. The neural network recovers rapidly (by $L = 3$) because the reward model aggregates per-transition estimates over longer segments and value iteration propagates them through the full transition structure; DPO improves monotonically but plateaus at its ∼95% ceiling, as it must learn the policy directly from trajectory comparisons without access to the transition model.

Two-stage RLHF separates preference estimation (a static econometric problem) from

[18] DPO learns only from (s, a) pairs visited in training trajectories generated by the random behavioral policy. It cannot propagate value to undervisited states the way value iteration does after learning a reward model, so states poorly covered by the behavioral policy remain suboptimal regardless of K.

[19] DPO fails catastrophically in gridworld because transitions are stochastic (10% slip probability) and rewards are transition-dependent. The same (s, a) pair yields different rewards depending on whether the agent slipped, so the DPO loss conflates policy quality with transition luck. In the job search model, accept/reject deterministically changes employment status, and only the 5% layoff probability introduces stochastic transitions.

[20] DPO's mean accepted amenity of 3.4 parallels the misspecified structural model's 3.0, though the mechanisms differ: the misspecified model ignores amenity variation by construction, while DPO underweights amenities because the random behavioral policy underrepresents high-amenity employed states in the training data.

[21] An online versus offline ablation for the neural network at $K = 1{,}000$ (20 seeds) shows comparable performance: online 73.92 ± 0.05, offline 73.99 ± 0.03 ($p = 0.09$). The random behavioral policy already provides diverse career trajectories covering the full wage-amenity space.

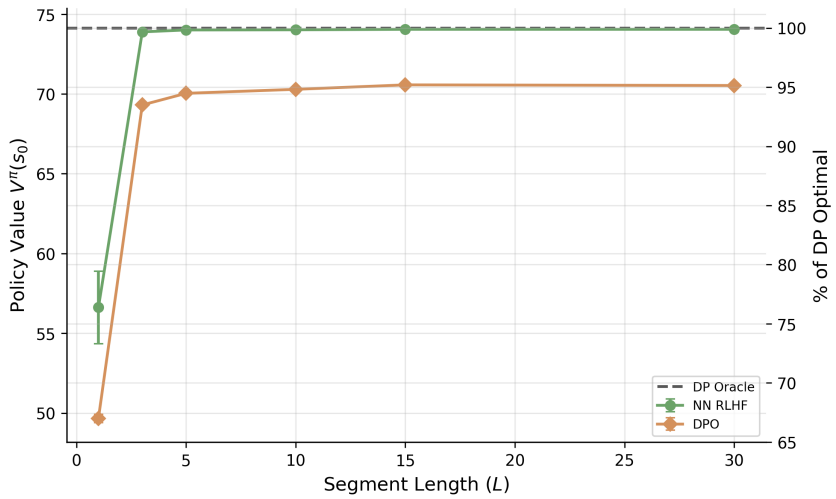

Figure 5: Policy value $V^\pi(s_0)$ versus segment length L at $K = 2{,}000$ for the neural network and DPO (20 seeds). The right axis shows the percentage of DP-optimal value.

dynamic programming (which exploits the known transition model); DPO conflates the two and forfeits the transition structure that economists typically have access to.[22]

[22]This advantage is specific to settings where the transition model is known or estimable; in domains without a tractable transition model, DPO's single-stage approach avoids compounding errors from reward model estimation.

References

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2052–2062. PMLR, 2019.

Fedor Iskhakov, John Rust, and Bertel Schjerning. Machine learning and structural econometrics: Contrasts and synergies. *The Econometrics Journal*, 23(S1):S81–S124, 2020.

Ying Jin, Zhuoran Yang, and Zhaoran Wang. Is pessimism provably efficient for offline RL? In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5084–5096. PMLR, 2021.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. *arXiv preprint arXiv:2205.11275*, 2022.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-Learning. In *International Conference on Learning Representations*, 2022.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-Learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Sascha Lange, Thomas Gabel, and Martin Riedmiller. Batch reinforcement learning. *Reinforcement Learning: State-of-the-Art*, pages 45–73, 2012.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9:815–857, 2008.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Paria Rashidinejad, Banghua Zhu, Cong Ma, Jiantao Jiao, and Stuart Russell. Bridging offline reinforcement learning and imitation learning: A tale of pessimism. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021, 2020.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Andrea Zanette. Exponential lower bounds for batch reinforcement learning: Batch RL can be exponentially harder than online RL. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*. PMLR, 2021.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario

Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.